

2026 Logic System Lab

邏輯系統實驗

Lab3 Report

	姓名	學號
組員 1	何家睿	E24146644
組員 2	張家榕	F84131251

組別： 01

[Question 1]

Prove that C_{out} derived from the two equations is the same

- Hint : $X + \bar{X}Z = X + Z$

- $A \oplus B = \bar{A}B + A\bar{B}$

1. $A \cdot B + (A \oplus B) \cdot C_{in} \Rightarrow A \cdot B + (A + B) \cdot C_{in}$

$$\begin{aligned} & A \cdot B + (A \oplus B) \cdot C_{in} \\ &= AB + A'BC + AB'C \\ &= B(A + A'C) + A(B + B'C) \\ &= B(A + C) + A(B + C) \\ &= AB + BC + AC = AB + (A + B) \cdot C_{in} \end{aligned}$$

2. $A \cdot B + (A + B) \cdot C_{in} \Rightarrow A \cdot B + (A \oplus B) \cdot C_{in}$

$$\begin{aligned} & A \cdot B + (A + B) \cdot C_{in} \\ &= AB + BC + AC \\ &= B(A + C) + A(B + C) \\ &= B(A + A'C) + A(B + B'C) \\ &= AB + A'BC + AB'C = AB + (A'B + AB') = A \cdot B + (A \oplus B) \cdot C_{in} \end{aligned}$$

[Question 2]

Explain the underlying logic behind this equivalence

$$A \cdot B + (A + B) \cdot C_{in} \equiv A \cdot B + (A \oplus B) \cdot C_{in}$$

Answer: By observing the truth tables of XOR and OR, we can see that the only difference between these two logic gates is when $A=1$ and $B=1$. In this entire function, there is another term AB . Because AB outputs 1 exactly in the case where $A=1$ and $B=1$, it already covers this specific condition. Thus, $(A \oplus B)$ can be replaced by $(A + B)$.

[Practice 1 ~ 4]

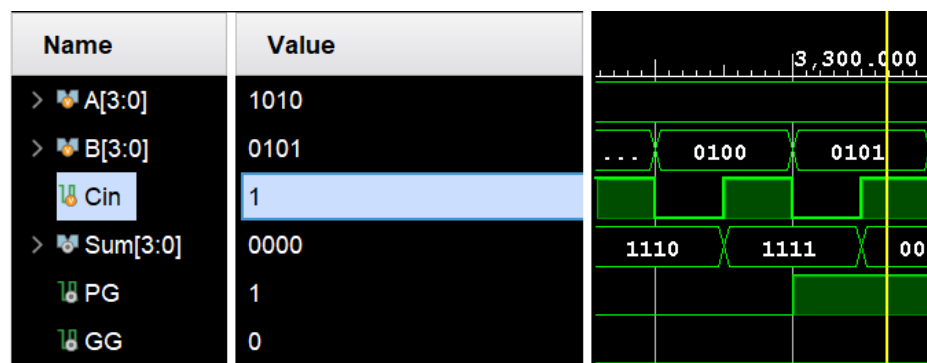
Enter Pass/Fail only (No screenshots required)

	pass or fail
RCA_4	<i>pass</i>
RCA_8	<i>pass</i>
CLL_4	<i>pass</i>
CLA_4	<i>pass</i>
CLA_64	<i>pass</i>
Adder_8_OV	<i>Pass</i>
ALU_8	<i>Pass</i>

[Question 3-1]

Screenshot the waveform for each case and explain why the values appear as they do

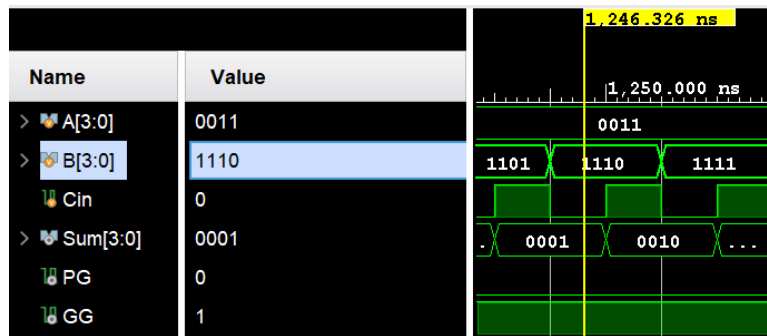
1. Case 1 : $A = 4'b1010, B = 4'b0101, C_{in} = 1'b1$



Explanation:

Note that $1010 + 0101 + 0001 = 10000$, which triggers an **overflow in a 4-bit** display (Sum=0000) . However, this specific value was captured by the **PG (Propagator-Generator)** unit, indicating a carry-out generated during the propagation process.

2. Case 2 : $A = 4'b0011, B = 4'b1110, C_{in} = 1'b0$



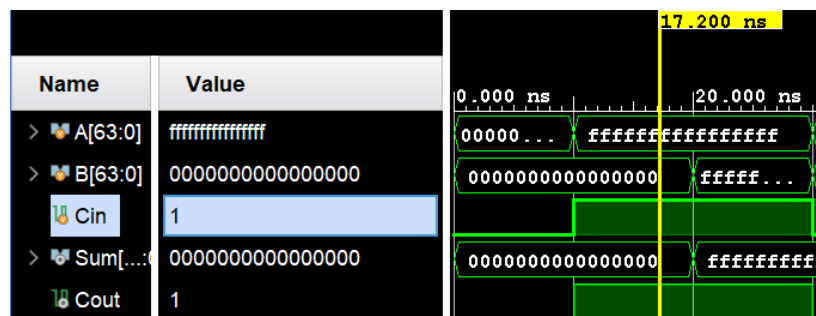
Explanation:

Note that $0011 + 1110 + 0000 = 10001$, which also results in a **4-bit overflow**. Consequently, the **Sum** = 0001. Nevertheless, this operation was recorded by the **GG (Group Generate)** signal, indicating a carry was generated.

[Question 3-2]

Screenshot the waveform for each case and explain why the values appear as they do

1. Case 1 : $A = 64'hFFFF_FFFF_FFFF_FFFF, B = 64'd0, C_{in} = 1'b1$



Explanation:

Similar to Q3-1, adding $FFFF, FFFF, FFFF, FFFF$ and $0000, 0000, 0000, 0001$ results in a 65-bit value ($1,0000, 0000, 0000, 0000$), which causes a **64-bit overflow**. Consequently, the **Sum** remains 0. However, this overflow bit is stored at **Cout**, which can then be propagated to subsequent modules.

[Question 4 : Overflow for Adder_8_OV]

1. Please describe when overflow occurs

Answer: An overflow occurs when both operands (A and B) have the same MSB (sign bit), but the resulting Sum has a different sign bit.

2. Please derive the Boolean expression of overflow

- $OV = (A[7] + B[7] + S'[7]) + (A'[7] + B'[7] + S[7])$
- *assign OV = ((AssignBit & BsignBit & ~SumsignBit) | (~AssignBit & ~BsignBit & SumsignBit));*
- *assign OV = (AssignBit == BsignBit) & (SumsignBit != AssignBit);*

[Question 5 : Overflow for ALU_8]

1. Please discuss whether overflow conditions need to be modified

Answer:

Overflow detection is essential for **signed addition** to ensure the result is correct. However, for **unsigned addition**, the overflow bit is simply the Carry-out, so it is often considered less of a 'problem' and more of just an extra bit.

2. Please derive the Boolean expression of overflow

Answer:

I use Adder_8_OV module , which Logic is:

$$OV = (A[7] + B[7] + S'[7]) + (A'[7] + B'[7] + S[7])$$

[Question 6]

Please explain CARRY4 in FPGA

1. How it works ?

CARRY4 是一個 4-bit 的硬體加速單元。它不像 RCA 要在那邊等 Gates 一個接一個傳遞訊號，而是利用 FPGA 內部預先連好的「專用進位線」(Dedicated Carry Chain)。它會同時接收 4 位元的資料，並配合旁邊的 LUT (查找表) 產生的 P、G 訊號，瞬間把進位算出來並丟給下一級。

2. Its Pros & Cons (優缺點) compared to RCA and CLA

Pros :

- **速度快：** 在 FPGA 裡，一般的佈線延遲很大；CARRY4 是快速通道」，專門給進位訊號跑，速度比我們自己用 Gates 寫出來的 RCA 快得多。
- **省算力：** 如果自己寫 CLA，會用到超多 LUT 資源，電路會變得很亂。用 CARRY4 的話，它是 FPGA 裡本來就有的閒置專用硬體，是人家寫好的 LUT。

Cons :

- **硬體通用性差：** 這是我覺得最明顯的缺點。CLA 的概念是通用的，不管在哪都能跑；但 CARRY4 是 Xilinx 特有的。如果今天換成別家的 FPGA，這套邏輯就得重寫，沒辦法直接搬過去。
- **位元固定：** 一次只能處理 4 bits。雖然可串聯，但如果要做一些很奇怪位元的運算，就會需要調整一下，不像自己寫的加法器那麼有彈性。

[Question 7]

Check if the synthesis tool utilized a CARRY4 primitive

for RCA & CLA, providing the reason why or why not

RCA_8:

Since my design uses **gate-level descriptions** for the basic units (Half Adder and Full Adder), Vivado treats the circuit as a collection of individual logic gates rather than a high-level addition operation. View to the schematic, proof that synthesis tool **does not** automatically replace them with the **CARRY4** primitive.

CLA_64:

For the same reason as the RCA_8, my CLA design utilizes gate-level descriptions for its two basic units: the **PFA** and **CLL**. Maybe the synthesis tool prefer to remaining this manual gate-level structure, it implements the design using **LUTs instead of the CARRY4** primitive.

[心得]

組員一 何家睿 E24146644

In this lab, I gained hands-on experience with adder circuits, which are the fundamental building blocks of a CPU's ALU. By building each module step-by-step in Verilog, I gained a great sense of accomplishment. The most rewarding part was implementing the ALU itself, as it allowed me to integrate everything I learned in this course into a single, functioning system.

組員二 張家榕 F84131251

在本週因為期中考等因素，所以我還沒開始動工，但依照先前我們兩個都喜歡各自做一份自己的報告來看，這次的 Lab3 4 我也會在截止日前完成一份的，很感謝助教用心做的教學影片，然後也感謝組員讓我可以比較沒有壓力的完成這次的實驗！